电商销量预测系统 · 测试报告 V1.0

项目	内容
文档版本	**V1.0**
对应阶段	EP-9 集成联调与测试（《项目开发实施方案 V1.0》§7.10）
验收依据	DOD-EP9（实施方案 §7.10，7 条） + WS-9-01~WS-9-09
测试对象	EP-0~EP-8 全部产出（算法层 + 后端 30 接口 + 前端 14 页面）
测试日期	2026-10-12
结论	**通过**（🔒 硬约束全部达成；7 项遗留问题已分级登记，无 P0）

📌 本报告是 DOD-EP9 第 7 条要求的交付物，含**通过用例 / 失败用例 / 修复记录 / 遗留问题清单**四部分。
所有数字均来自落盘证据文件（见 §6），不是人工抄录。

⚠ **数字时点说明（2026-10-27 补记，不改原值）**

本报告是 **2026-10-12 成稿的历史快照**，其中所有计数（接口数、校验套数、
断言语数、索引数等）都**只代表那一刻**。此后代码有若干演进，
按「历史报告不改数字、只加时点说明」的纪律，原文一律保留，现值看这里：

项	本报告成稿时	现值（2026-10-27）	权威来源
接口	30 个（API-01~API-30）	**31 条路径 / 33 个操作**（+ API-19a 历史导出）	outputs/电商销量预测系统_openapi.json
前端页面	14 个	**14 个**（未变）	ls web/src/pages/*.tsx
校验脚本	EP-9 时 35 套 / EP-10 收口 38 套	**40 套**；scripts/ 下 **39** 个校验脚本全部登记，CHAIN **41** 条	python scripts/tasks.py list

| verify_ep9.py 断言 | 52 项 | **58 项** | python scripts/verify_ep9.py |

| 数据库对象 | 8 表 / 121 列 / 12 索引 | **9 表 / 125 列 / 14 索引**（+ revoked_tokens） | python scripts/verify_schema.py |

| pytest | 未纳入（EP-9 时尚无 pytest） | **128 条** | python -m pytest tests/ -q |

若需要「此刻是什么样」，请以表中「权威来源」列的脚本输出为准 ——

不要回头修改本文的任何数字：那会让这份报告变成伪造的记录。

目 录

1. 测试环境 4

1.1 被测版本 4

1.2 环境 4

1.3 测试方法 5

1.4 浏览器工具链：本机只有 Edge，怎么跑满三款内核 5

2. 通过用例 6

2.1 算法层基线回归（WS-9-09 / TEST-1） 6

2.2 WS-9-01 主干路径端到端 7

2.3 WS-9-02 分支场景 B1~B14 8

2.4 WS-9-03 边界与异常 B-01~B-25（技术文档 §9.4） 9

2.5 WS-9-04 异常操作不产生脏数据 10

2.6 WS-9-05 性能测试（技术文档 §9.7 全 7 项） 11

2.7 WS-9-06 兼容性（真实浏览器走查） 13

2.8 WS-9-07 无障碍审计 14

2.9 WS-9-08 契约一致性 15

2.10 WS-9-09 校验链全绿 16

2.11 上游阶段校验链全量回归（EP-3~EP-8） 17
3. 失败用例 19
3.1 产品缺陷类（5 项，全部已修复） 19
3.2 校验脚本自身缺陷类（25 项，全部已修复） 21
3.3 文档勘误类（3 项） 25
4. 修复记录汇总 26
5. 遗留问题清单 27
P1（重要，需在 EP-10 说明或补救） 27
P2（二期） 28
P3（不排期） 29
6. 证据文件清单 29
7. DOD-EP9 逐条对照 30

1. 测试环境

1.1 被测版本

项	值
模型版本	`lstm_v1`（`outputs/models/lstm_v1.pt`，window=21，hidden=32，params=31,729）
推理设备	`cuda`（NVIDIA MX450，2048 MiB，CC 7.5）
后端	FastAPI + Uvicorn，30 个接口（`API-01`~`API-30`）
前端	React 18 + Vite 5.4.21 + Ant Design 5 + ECharts
数据库	SQLite（WAL），`code/db/schema.sql`

1.2 环境

项	值
操作系统	Windows 10
CPU / 内存	8 逻辑核 / 23.8 GB
GPU	MX450 2048 MiB（驱动 576.28，CUDA 12.9）
Python	3.13.12（conda 环境 `deploy`，torch 2.9.1+cu128）
Node	22.22.2
浏览器	Edge 155.0.4283.45（系统自带）；Chrome 155.0.8059.39（Chrome for Testing，便携）；Firefox 157.0.1（Gecko，便携 + WebDriver BiDi）；Safari **Windows 无发行版**（见 §5 `P1-01`）
服务端口	API 8021 / 前端 dev 5199 / 前端 preview 4173

⚠ **环境变量的两个坑（本轮实测，已写入脚本注释）**：

① 本机设了 `HTTP_PROXY=http://127.0.0.1:51141`，无头浏览器会继承它并**代理本地地址**，

表现为「页面全白、断言全失败」；浏览器启动参数必须带 `--no-proxy-server`。

② `vite.config.ts` 的代理目标默认是 `8000`，而后端跑在 `8021` —— 不显式设

`VITE_API_TARGET` 时页面能渲染但**每个接口都失败**（页面空着、后端日志一条请求都没有）。

1.3 测试方法

采用**脚本化回归 + 真实浏览器走查**，不使用人工点检：

层	手段	数量
算法层	`self_check.py` / `eval_report.py`	2 套
后端接口	`TestClient` 全量回归（独立临时库，不污染 `data/app.db`）	12 套
前端/文档静态守卫	AST / 正则 / 结构断言	10 套
真实浏览器	Edge / Chrome（原生 CDP）+ Firefox（WebDriver BiDi），不依赖 playwright	7 套
契约比对	前后端双向结构比对	1 套
合计	见 §2.10 校验链清单	**35 个脚本**

🔒 **运行纪律（本轮发现，已写入每个脚本的文件头）**：

平台的批量删除保护按「**单次工具调用的累计删除数**」计数（阈值 50）。

一条命令里 for 循环串跑多个校验脚本会让前面脚本的临时目录清理量

累积到后面脚本头上，被判为批量删除后脚本**直接退出**，日志只留一行

`STATUS: FAILED` —— 看起来像代码缺陷，实际是平台的保护机制。

故：**每个校验脚本单独占一条命令。**

1.4 浏览器工具链：本机只有 Edge，怎么跑满三款内核

`DOD-EP9` 要求四浏览器。本机只有系统自带的 Edge，缺的两款按下面的方式补齐

（**全部解包到工作区 `data/tmp/browsers/`，不写注册表、不建快捷方式、不污染系统**）：

浏览器	获取方式	体积
Chrome	npmmirror 的 `chrome-for-testing` 镜像 —— 官方为**测试**发布的独立二进制、原生支持 CDP，解压即用	下载 207 MB / 解压 438 MB
Firefox	npmmirror 的官方 `Firefox Setup 157.0.1.msi` → **7-Zip 两层展开**（见下）	下载 92 MB / 解压 350 MB
Safari	——	✕ 无法获取：Windows 平台无 Safari 发行版

🟤 **Firefox 便携化的坑**：官方 MSI 其实是个 **wrapper**，真正的安装器在内嵌的 `Binary.WrappedExe`（93 MB）里，产品文件一个都没有。`msiexec /a`（管理员安装）在这种结构下**解不出任何文件** —— 只会把 msi 自己复制到 `TARGETDIR`，而**日志与退出码仍返回 0**，极具迷惑性。可行路径是 7-Zip 两层展开：`7z x "<msi>"` → 得 `Binary.WrappedExe` → `7z x Binary.WrappedExe` → 得 `core/firefox.exe`。（本机 npm 镜像若不可达，Chrome/Firefox 的版本号取自 `chrome-for-testing` 的 `last-known-good-versions.json` 与 Mozilla `latest` 重定向，均**联网核过**。）

---

2. 通过用例

2.1 算法层基线回归（WS-9-09 / TEST-1）

项	结果	门槛	判定
`self_check.py` 自检	**69 / 69**	69/69（`TEST-1` 基线）	☑
加权 MAPE	**14.26%**	≤ 20%	☑
加权 MAE 相对移动平均基	**26.42%**	≥ 15%	☑

项	结果	门槛	判定
线降低			
加权 RMSE 相对移动平均基线降低	**27.52%**	≥ 15%	✅
达标序列占比	**83.95%**（68/81）	≥ 60%	✅
分级 MAPE	L1 14.26% / L2 60.33%	—	披露

分级覆盖：L1 = 81 / L2 = 32 / L3 = 0（示例数据集 `demo_sales.csv` 不含 L3 实例）。

> ⚠ 正因为示例数据的 **L3 = 0**，「不可预测」这条分支在常规测试中永远不会执行 ——
> 这是 `B-11` / `B-12` 必须**构造数据**触发的原因（`TEST-2`）。
>
> 📌 回归结果与 `V1.1` 开发方案记录的数字**逐项一致**，且 `timing_ms` 走
> 「基准冻结」机制（本次实测 5,464 ms 另存 `eval_timing_last.json`，报告仍记 7,873 ms），
> 故前端展示数据无需重新同步。

2.2 WS-9-01 主干路径端到端

`scripts/verify_ep9_perf.py::case_main_path`，全程无人工介入，逐步计时：

步骤	耗时	结果
注册	447 ms	`code=0`
登录	447 ms	拿到 Token
上传数据集（6 商品 × 2 仓 × 120 天）	106 ms	`code=0` 校验通过
`API-09` 详情与耗时预估	26 ms	带 `estimate_by_horizon`（3 档）
`API-12` 建任务（h=14）	1,177 ms	`status=succeeded`
`API-13` 状态	12 ms	进度 100%
`API-14` 结果	40 ms	指标齐备
`API-15` 明细分页	16 ms	`total=168`

步骤	耗时	结果
API-16 导出 CSV	21 ms	7 列契约
全程合计	**2,293 ms**	—

2.3 WS-9-02 分支场景 B1~B14

按《开发方案》§2.2 的 14 条分支场景，由各阶段校验脚本覆盖验证：

分支	场景	覆盖脚本	结果
B1	游客链路（上传→参数→执行→结果）	verify_ep4_browser / verify_ep9_browser	☑
B2	未登录访问受限页 → 跳登录	verify_ep7_browser / verify_ep9_browser	☑
B3	断档 → 用户显式选择处置	verify_ep4_api (B-05)	☑
B4	序列数超限	verify_ep4_api (B-06)	☑
B5	数据量不足	verify_ep5_api	☑
B6	无启用模型	verify_ep9_api (B-18)	☑
B7	超 60 s 软失败	verify_ep9_api (B-16) / perf	☑
B8	CUDA OOM → 回退 CPU	verify_ep9_api (B-17)	☑
B9	任务失败后重试	verify_ep7_api (原参数重试)	☑
B10	软删除与可追溯	verify_ep7_api / verify_ep8_api	☑
B11	越权访问	verify_ep9_api (B-20/B-21)	☑
B12	401 跳登录并保留草稿参数	verify_ep5_browser	☑
B13	跨任务对比（2~4 个）	verify_ep7_api (B-24/B-25)	☑
B14	管理端越权与非管理员不可见	verify_ep8_browser	☑

2.4 WS-9-03 边界与异常 B-01~B-25（技术文档 §9.4）

`scripts/verify_ep9_api.py`，**25 / 25 条通过，114 条断言，总 125 项通过 / 0 失败**（耗时 96.2 s）：

#	用例	断言数	结果
B-01	缺 `warehouse_id` 列 → `40001` + `missing_columns`	3	✅
B-02	缺 `sales` 列 → `40001`	2	✅
B-03	日期不可解析 → `40002` 带行号与原始值	6	✅
B-04	销量负值 → `40003` 指出行号	3	✅
B-05	时间断档 → `40005` + 未处置不继续	5	✅
B-06	序列数超限 → `40004` 提示当前值	2	✅
B-07	文件超 20 MB → `40007` 无磁盘残留	4	✅
B-08	行数超 50 万 → `40008`	1	✅
B-09	空文件 → `40010`	1	✅
B-10	非 CSV 改名 → **不在扩展名阶段放行**	3	✅
B-11	🔒 **L3 序列**：落 1 行占位（值=历史均值），行数 ≠ horizon，不计入主指标	14	✅
B-12	🔒 **L2 序列**：`l2_mape` 为 `null` 而非 `NaN`	5	✅
B-13	全零序列：除零保护，`mape` 无 `Inf`	4	✅
B-14	单序列数据：正常完成，`series_total=1`	6	✅
B-15	中文与特殊字符（含逗号、引号、空格）全链路正常	4	✅
B-16	🔒 超 60 s 任务：`E_TIMEOUT` + 已完成阶段，**非 5xx**	10	✅

#	用例	断言数	结果
B-17	🔒 CUDA OOM：自动回退 CPU 且 `device_fallback=1` **落库**	7	✅
B-18	无启用模型：配置漂移必须**启动失败**	5	✅
B-19	并发 3 任务：全部完成，无 `database is locked`	4	✅
B-20	越权访问 → `40301`（**不是 404**）	5	✅
B-21	导出越权 → `40301`	2	✅
B-22	路径穿越探针 5 条：**均未读出工作区外文件**	4	✅
B-23	SQL 注入：表仍在，参数化查询转义	6	✅
B-24	对比数量非法（1 个 / 5 个）→ `40012`	3	✅
B-25	对比步长不一致 → 返回结果 + `warnings` 提示	3	✅

🔒 `B-16` / `B-17` 按 `TEST-3` **实际模拟**（注入真实 `RuntimeError("CUDA out of memory...")` 走真实代码路径；极限规模用独立子进程跑 1000 序列），不是代码审查。

🔒 `B-11` / `B-12` 按 `TEST-2` **构造稀疏夹具**触发：3 个商品 × 不同仓覆盖度（L1 120 天 / L2 跨仓合计 110 天 / L3 仅 5 天 / 全零序列），一次任务同时覆盖 `B-11`、`B-12`、`B-13`、`B-15`。

2.5 WS-9-04 异常操作不产生脏数据

把「脏数据」定义为**六条可判定事实**，逐条查库断言（`case_dirty_data`）：

#	事实	实测	结果

#	事实	实测	结果
①	无孤儿行（predictions/metrics.task_id 均存在）	0 / 0	✅
②	无悬挂任务（无 status='running' 遗留）	0	✅
③	无半成品（非 succeeded 任务不带预测行）	0	✅
④	无重复键（任务×(仓,商品,日期) 唯一）	0	✅
⑤	上传目录无 0 字节残留	0	✅
⑥	无越界归属（任务归属 = 数据集归属）	0	✅

操作层面的三条断言：

操作	预期	实测	结果
二次上传同一文件	生成两个**独立**数据集行，落盘路径不同，互不覆盖	[4, 5]，路径哈希不同	✅
重复提交同一请求	生成两个独立任务，各 7 行，互不污染	[4, 5]，各 7 行	✅
任务完成后反复读取（15 次含导出）	读操作无副作用，计数不变	tasks/datasets/predictions/metrics 快照完全一致（5 / 5 / 1,778 / 5）	✅

2.6 WS-9-05 性能测试（技术文档 §9.7 全 7 项）

#	测试项	门槛	实测	结果
1	端到端耗时（示例数据 113 序列，h=14）	🔒 ≤ 60 s	**11,855 ms**（基线 7,873 ms 的 1.51×，TEST-4 警戒线 3.0×）	✅

#	测试项	门槛	实测	结果
2	极限规模（1,000 序列 × h=30）	≤ 60 s 或**正确软失败**	软失败：`E_TIMEOUT` + `completed_stage=metrics` + 无半成品 + HTTP 200	✅
3	并发 3 任务	全部成功、无锁冲突、**各自** ≤ 60 s	全部 `succeeded`；`duration_ms` = **32,207 / 37,610 / 37,513 ms**（墙钟 32,252 / 37,653 / 37,567 ms）；`database is locked` 命中 **0** 次	✅
4	首屏	≤ 3 s（`NFR-PERF-06`）	**FCP 中位数 504 ms**（冷加载 748 / 含缓存 152 / 504）；`loadEventEnd` 最大 400 ms	✅
5	预览接口（20 行）	≤ 1 s（`NFR-PERF-07`）	**56 ms**（3 次取最慢）	✅
6	历史列表（单页 20 条）	≤ 1 s（`NFR-PERF-08`）	**61 ms**（3 次取最慢，`total=1 items=1` 非空表）	✅
7	GPU 回退（强制 CPU）	≤ 60 s（`NFR-PERF-10`）	**23,489 ms**，`device=cpu` 已确认	✅

极限规模的细节（第 2 项）：

项	值
夹具	200 商品 × 5 仓 × 120 天 = **1,000 序列**，3.3 MB / 120,001 行
上传	560 ms，`series=1000` 校验通过
任务终态	`status=failed`，`error_code=E_TIMEOUT`，`completed_stage=metrics`
是否 5xx	否（HTTP 200 + `code=0` + 终态软失败）
半成品	该任务 `predictions` 行数 = 0（未落半成品）
耗时	250,302 ms（远超 60 s 预算 → 按设计软失败）

📌 DOD-EP9 原文为「极限 1000 序列 × horizon=30 **通过或正确软失败**」，

此处走软失败路径且**五项判据全部满足**（非 5xx / 有错误码 / 有已完成阶段 /

无半成品 / 用户可理解），故判定通过。

⚠ 但这意味着 60 s SLA 在极限规模下**靠软失败兜底而非性能达标**，已登记为 P2-01。

2.7 WS-9-06 兼容性（真实浏览器走查）

scripts/verify_ep9_browser.mjs，**同一套用例在 3 款浏览器内核上各跑一遍**，全部 **109 / 109 通过**：

浏览器	版本	驱动通道	结果
Edge	Chromium 155.0.4283.45	原生 CDP	✅ **109 / 109**
Chrome	Chromium 155.0.8059.39（Chrome for Testing）	原生 CDP	✅ **109 / 109**
Firefox	Gecko 157.0.1	**WebDriver BiDi**	✅ **109 / 109**
Safari	WebKit	——	⚠ **BLOCKED**：Windows 平台无发行版（见 P1-01）

视口	覆盖页面	断言
1366×768	首页 / 登录 / 注册 / 上传（游客）/ 参数（游客）/ 工作台 / 上传 / 参数 / 结果 / 历史 / 详情 / 对比 / 看板 / 管理（14 页）	无横向溢出 + 内容非空 + 非异常态（+ 登录态页「停在预期路径」）
1600×900	同上 14 页	同上

项	结果
横向溢出	**0 例**（28 页 × 2 视口 × 3 浏览器，scrollWidth ≤ innerWidth）
空白页	**0 例**（结果页文本长度 4,005；参数页 1,759；详情页 2,746）
停在预期路径（守卫通过）	**18 / 18**（登录态与管理员页全部未被静默踢回 /login）

项	结果
🔒 浏览器控制台错误	**0 条**（含未捕获异常；3 款浏览器各自 0）
🔒 首屏 FCP（生产构建口径）	Edge **140 ms** / Chrome **140 ms** / Firefox **367 ms**（均 ≤ 3 s，样本 3/3 有效）
截图证据	`outputs/ep9-shots/{narrow,wide}/`（Edge）、`outputs/ep9-shots/{chrome,firefox}/`
报告证据	`outputs/ep9-shots/ep9_browser_report{,_chrome,_firefox}.json`

🔧 **为什么这 3 款是「同一套用例」而不是「各自点一遍」**：Firefox ≥129 已**移除 CDP**，只提供 WebDriver BiDi。若为它另写一套用例，就等于把 300 多行断言复制成两份，从此必须同步维护 —— 迟早会漂移成「Edge 测到的和 Firefox 测到的不是同一件事」。

因此脚本改为**在 BiDi 之上加一层 CDP 兼容垫片**（`CdpOverBidi`）：对外暴露与 `class CDP` 完全相同的 `send / eval / on`，内部只翻译本脚本真正用到的 6 个方法。

全部用例代码一行没改。

跑通过程中撞到 4 个只在实测中才会暴露的坑（已写入 `P1-01` 的处置记录）：

① Firefox 没有 `/json/version` HTTP 发现端点，BiDi 的 WebSocket 直接开在调试端口上；

② 且**握手路径是 `/session`**，根路径返回 `200 + HTML`（Node 只报无细节的 `non-101`）；

③ `session.status` 的 `{ready:true}` **不代表会话已建立**，必须显式 `session.new`；

④ BiDi 的 `script.evaluate` 返回 RemoteValue（对象被拆成 `[[key, RemoteValue], …]`），**必须反序列化**，否则字段全是 `undefined`、断言会**静默地全绿或全红**。

🔒 **「停在预期路径」这条断言是本轮新增的。**原因：三条守卫链（`RequireAuth` / `RequireAdmin` / 游客链路）在凭据失效时会**静默跳 `/login`**，而登录页同样「无溢出、有内容、非异常态」—— 三条断言全部通过，整轮走查却一条业务页都没测到。

「断言全绿但什么都没测」是走查脚本最危险的失效模式，故补此断言钉死它。

2.8 WS-9-07 无障碍审计

`python scripts/a11y_audit.py` → **退出码 0**（证据：`outputs/ep9_a11y_log.txt`）：

项	值
审计项	53
PASS	51
FAIL	**0**
豁免	2（`--border-strong` 1.82:1 与 `--border` 1.32:1，均为**装饰性**分隔线/描边，非承载信息的图形边界；脚本内置豁免理由）
深浅双主题合计 FAIL	**0**

审计对象是**前端实际生效的** `web/src/theme/tokens.css`（oklch 还原为 sRGB 后按 WCAG 2.1 相对亮度公式实算），不是脚本内硬编码的副本 —— 因此不存在「审计通过但线上不达标」的空档。

2.9 WS-9-08 契约一致性

`scripts/verify_ep9_contract.py`，**92 / 92 通过**。四组**双向**比对：

组	比对内容	规模	结果
A	`code/backend/schemas.py` ↔ `web/src/types/index.ts` 字段名	后端 62 个模型 / 前端 51 个 interface	✅ 字段双向一致
B	OpenAPI `paths` ↔ `client.ts` 调用点	32 条路由 / 32 个调用点	✅ 双向无差（无死接口、无 404 隐患）
C	pydantic `Literal` ↔ TS 字面量联合类型	`TaskStatus` / `GranularityLevel` / `TaskErrorCode` / `UserRole` / `Horizon` / 错误码表（26 个）	✅ 取值域全部一致
D	后端常量 ↔ TS `LIMITS` / `THRESHOLDS` / 列序常量	上限 7 项 + 阈值 3 组 + `REQUIRED_COLUMNS` / `EXPORT_COLUMNS` /	✅ 全部同源

组	比对内容	规模	结果
		VALIDATE_ORDER / TOKEN_VERSION_DEFAULT	

🔒 **本组的关键机制**：MODEL_TO_TS 是一张**必须逐条登记的映射表**，两端任一侧新增模型/接口而不登记即失败。因此「忘了同步」在结构上不可能静默通过 —— 这是它与 verify_schemas.py（只查后端一侧）的本质差别。

📌 审查中显式登记了一处**同名不同义**（本轮发现）：后端存在两个都叫 EXPORT_COLUMNS 语义的常量 —— models/prediction.EXPORT_COLUMNS 是**库列序**，result_service.EXPORT_HEADER 才是**导出文件表头**（列名刻意不同，不向业务方泄漏 forecast_date 这类实现术语）。前端常量对应的是后者。该断言已固化为「库列序与业务表头**必须不同**」，防止后人「顺手统一」掉。

2.10 WS-9-09 校验链全绿

scripts/verify_ep9.py —— **52 项通过 / 0 失败**，退出码 0，报告写入 outputs/ep9_chain_report.json。做三件事（均为静态，理由见下）：

① **校验链清单守卫** —— 35 个脚本逐个断言存在；**反向**断言 scripts/ 下任何 verify_* / a11y_audit / self_check 若未登记即失败（防止新增脚本后忘跑）。

② **§9.4 / §9.7 覆盖度守卫** —— 技术文档 §9.4 表格含全部 B-01~B-25、verify_ep9_api.py 亦覆盖同编号（脚本与文档同步）；技术文档 §9.7 表格仍为 7 行。

③ **DOD-EP9 交付物守卫** —— 测试报告 / 性能报告 / 契约报告 / a11y 日志 / 走查截图目录齐备，且测试报告含四个必备章节。

校验链清单（按类别）：

类别	数量	脚本
算法层	2	`self_check.py`、`eval_report.py`
数据/模型静态	3	`verify_orm.py`、`verify_schema.py`、`verify_schemas.py`
前端静态	3	`verify_types.mjs`、`verify_tokens.py`、`verify_prototype.mjs`
文档	2	`verify_doc_refs.py`、`verify_doc_sql.py`
认证	1	`verify_auth.py`
阶段静态	6	`verify_ep3.py`~`verify_ep8.py`
阶段接口	6	`verify_ep4_api.py`、`verify_ep5_api.py`、`verify_ep5_equiv.py`、`verify_ep6_api.py`、`verify_ep7_api.py`、`verify_ep8_api.py`
浏览器	6	`verify_ep3_browser.mjs`~`verify_ep8_browser.mjs`
无障碍	1	`a11y_audit.py`
`EP-9` 新增	**5**	`verify_ep9_api.py`、`verify_ep9_perf.py`、`verify_ep9_browser.mjs`、`verify_ep9_contract.py`、`verify_ep9.py`

⚠ **本脚本不亲自实跑那些校验脚本**（除自身），原因见 §1.3 的批量删除保护纪律 ——

若用 `subprocess` 串起来跑，删除量会累积到同一次调用上导致后续脚本被平台中断。

「全绿」的事实由 **§2.11** 逐条给出（本轮全部**实跑**，不是静态推断）。

2.11 上游阶段校验链全量回归（EP-3~EP-8）

`EP-9` 期间修改了源码（`F-01` 改 DAO/服务层、`F-02`/`F-05` 改错误码口径），

故对**上游全部校验脚本**做一次全量回归 —— 只跑 `EP-9` 自己的脚本不足以证明「没有回归」。

前置：后端 `8021`（独立库 `data/tmp/ep9_chain/`）+ 前端 dev `5199`（`VITE_API_TARGET=8021`）；

隔离库为空，故先用 `scripts/_ep9_make_fixture.py` 造一个**游客成功任务**（32 序列 = 20 条 `L1` + 12 条 `L2`）。

类别	脚本	结果
算法层	`self_check.py`	**69 / 69**
算法层	`eval_report.py`	四条硬指标与基线**完全一致**（`14.26%` / `↓26.42%` / `↓27.52%` / `83.95%`）
数据/模型静态	`verify_orm.py`	全部通过（含 10 个索引、3 条架构红线）
数据/模型静态	`verify_schema.py`	全部通过（含种子幂等性）
数据/模型静态	`verify_schemas.py`	**61 / 61**
前端静态	`verify_types.mjs`	全部通过（`types.ts` 612 行 / `api-client.ts` 266 行）
前端静态	`verify_tokens.py`	通过（`oklch(` 81 处、`hex` **0** 处）
前端静态	`verify_prototype.mjs`	13 页渲染，异常 **0** 页
文档	`verify_doc_refs.py`	编号引用无悬空
文档	`verify_doc_sql.py`	8 表 **121 列**与设计文档一致
认证	`verify_auth.py`	**39 / 39**
阶段静态	`verify_ep3.py`	**29 / 29**
阶段静态	`verify_ep4.py`	**77 / 77**
阶段静态	`verify_ep5.py`	**50 / 50**
阶段静态	`verify_ep6.py`	**65 / 65**
阶段静态	`verify_ep7.py`	**89 / 89**
阶段静态	`verify_ep8.py`	**108 / 108**
阶段接口	`verify_ep4_api.py`	**119 / 119**
阶段接口	`verify_ep5_api.py`	**60 / 60**（修复 `F-05` 后）
阶段接口	`verify_ep5_equiv.py`	**3 / 3**（`horizon=14/30` 逐位等价）
阶段接口	`verify_ep6_api.py`	**105 / 105**
阶段接口	`verify_ep7_api.py`	**70 / 70**
阶段接口	`verify_ep8_api.py`	**96 / 96**
浏览器走查	`verify_ep3_browser.mjs`	**47 / 47**（修正 `T-21` 后）
浏览器走查	`verify_ep4_browser.mjs`	**173 / 173**
浏览器走查	`verify_ep5_browser.mjs`	**41 / 41**（修正 `T-22` 后）
浏览器走查	`verify_ep6_browser.mjs`	**42 / 42**（补夹具后）

类别	脚本	结果
浏览器走查	verify_ep7_browser.mjs	**57 / 57**
浏览器走查	verify_ep8_browser.mjs	**61 / 61**
无障碍	a11y_audit.py	退出码 **0**（53 项，PASS 51 / FAIL 0 / 豁免 2）
EP-9	verify_ep9_api.py	**125 / 0**
EP-9	verify_ep9_perf.py	**47 / 47**
EP-9	verify_ep9_browser.mjs（Edge / Chrome / Firefox **各跑一遍**）	**109 / 109 × 3**（+ 1 BLOCKED：Safari）
EP-9	verify_ep9_contract.py	**92 / 92**
EP-9	verify_ep9.py	**52 / 52**

📌 **本轮全量回归的收获**：发现 **1 个真实产品缺陷**（F-05：API-13 越权码与全局口径不一致）与 **4 个测试工具缺陷**（T-21~T-24）。若只跑 EP-9 自有脚本，F-05 会漏网 —— 这是「上游回归」不可省略的直接证据。

---

3. 失败用例

本节列出**测试过程中真实出现过的失败**与处置。所有失败项最终均已消除；未消除的进 §5 遗留问题清单。

3.1 产品缺陷类（5 项，全部已修复）

#	现象	根因	影响面	处置
F-01	B-17（CUDA OOM 回退）断言失败：库里 device='cuda',	task_dao.mark_succeeded/mark_failed **不写** device/device_fallback；mark_running 落的是**开工时**的设备，	重启服务或从历史列表重开任务后，「本次是降级跑	mark_succeeded/mark_failed 新增两参数（is not None 才覆盖，避免抹成

#	现象	根因	影响面	处置
	device_fallback=0，实际已降级 CPU	而 OOM→搬 CPU→重跑成功发生在开工**之后**。降级事实只活在内存态 _Progress 里	的」这一事实**永久丢失**；用户与运维都无法察觉结果来自 CPU 回退	NULL）；predict_service 两处调用传值；并把 OOM 记账**提前到 CPU 重跑之前**（若重跑也抛异常，兜底出口拿到的才是「已降级」的真实状态）
F-02	B-20（越权访问）断言失败：返回 40401 而非 40301	result_service._owned_task 对「任务存在但不属于你」直接抛 TASK_NOT_FOUND	违反 NFR-SEC-05（PRD **P0**）：越权返回 404 会让攻击者**无法区分**「不存在」与「无权限」，看似更安全，实则与 datasets、history/compare 两处口径不一致，前端也无法给出正确引导（「去列表重选」还是「无权访问」）	改抛 FORBIDDEN(40301)；history_service.soft_delete 拆成三码：不存在→40401、非本人→40301、本人重复删除→**幂等成功**。三个码不可合并（合并任意两个，前端都无法正确引导）
F-03	B-11（L3 序列）断言失败：实现「落 1 行占位」，而技术文档写「该序列**不写入** predictions」	**三方口径冲突**：PRD FR-MOD-08（L3 输出历史均值）+ FR-VIS-04（三类标签均可出现）vs 技术文档 API-15-1/B-11（不落行）vs 实现（落 1 行占位）	若按技术文档改实现，前端结果页将无法用统一查询展示 L3（FR-VIS-04 要求三类标签都可出现）；若按 PRD 改，需同步三处文档	按「**功能范围以 PRD 为准**」的仲裁规则判定**实现在、文档错**：修订技术文档 API-15-1（整段重写 + 冲突对照表）、§9.4 B-11 判据、§6.4 前端明细表 L3 处理 行
F-04	契约审查发现 models/prediction.py 文件头写「导出列序（FR-VIS-05）由 EXPORT_COLUMNS 固定」	注释与实现不符：该常量是**库列序**（forecast_date/predicted_value…），真正的导出表头是 result_service.EXPORT_HEADER（date/forecast_sales…），两者**刻意不同**	误导性注释：后人若照此「统一」，会把实现术语（forecast_date）泄漏给业务方	重写该文件头注释与常量注释，显式写出两者的区别与「不要混用」的告警；并在契约脚本中固化为断言
F-05	verify_ep5_api.py ⑨ 断言失败：登录用户读**他人任务状态**返回 40401（期望 40301）	predict_service.get_status（API-13）用 task_dao.get_owned 一把过滤，把「任务不存在」与「越权」**合并成同一个 40401**。F-02 收敛越权口径时只改了 result_service（结果）与	同一个「别人的资源」在**结果接口**说 403、在**状态接口**说 404 —— 前端无法统一处理（该	改为与 result_service._owned_task 同款的**三步判定**：不存在 / 已软删除 → 40401，存在但不属于调用者 → 40301；get_status docstring 同步

#	现象	根因	影响面	处置
		`history_service`（删除），**漏了任务状态查询这条路**	引导「去列表重选」还是「无权访问」），也违背 PRD `NFR-SEC-05`（P0）「越权返回 403 而非 404（避免资源探测）」。**这条只有跑上游全量回归才会暴露**（见 §2.11）	写明三种出口与「顺序不可颠倒」的理由

3.2 校验脚本自身缺陷类（25 项，全部已修复）

📌 **为什么要单独统计「测试工具的缺陷」**：本轮 **24 个**脚本缺陷全部是断言/取数/夹具写错或口径过时，**没有一项是产品问题。**若不把它们与产品缺陷分开，测试报告会严重高估风险，同时也会掩盖真正的问题（`F-01`/`F-02`/`F-05` 就混在里面）。

#	脚本	缺陷	形态	处置		
T-01	全部 API 脚本	一条命令串跑 6 个脚本，触发平台批量删除保护	**假失败**：日志仅一行 `STATUS: FAILED`	改为每脚本单独一条命令；纪律写入所有脚本文件头		
T-02	`verify_ep9_api`	`dataset_id_of()` 只认 `data.dataset_id` 一种响应形状（成功时是 `data.dataset.id`）	5 条用例连锁「未执行」	两种形状都覆盖，教训写进函数 docstring		
T-03	`verify_ep9_api`	`B-03` 夹具用了 `2015/01/02`、`20150103` —— 这些是**可解析**的等价写法，实现放行是对的	断言方向错	改用真正不可解析值（`2015/13/45`、`20150103T99`、未知、空串）+ 增加「标准格式必须通过」对照组		

#	脚本	缺陷	形态	处置		
T-04	verify_ep9_api	B-10 判据写死「表头完整性」，实测真 xlsx 改名 .csv 落在「大小/行数」	跨阶段硬比	按实测收敛为 fail_stage ∈ (大小/行数，表头完整性) 且 code ∈ (40001, 40010)；同时把「40010 文案对二进制不够贴切」登记为 P3-02		
T-05	verify_ep9_api	B-11 断言按旧文档写「L3 不落行」	与裁决后的口径不符	改为「行数 == 1、值 == 历史均值 5.0、行数 ≠ horizon、可筛出、不计入主指标」		
T-06	verify_ep9_api	B-11 断言 series_evaluated == L1	全零序列 MAPE 分母为 0 → 被合理剔除，故实为 ≤	改 evaluated ≤ L1，并追加 evaluated == 2（能捕获夹具变化）		
T-07	verify_ep9_api	B-20 用新注册用户访问他人数据集 → 先被数据集归属拦下	前置失败	改用数据集所有者的 token 建任务		
T-08	verify_ep9_perf	并发用例读 CreateTaskResponse.elapsed_ms	该字段属 TaskStatusResponse；.get() 返回 None → 补 0 → 断言恒假	改读 duration_ms，并把「跨接口取同名字段前先确认归属」写进注释		
T-09	verify_ep9_perf	历史列表用例收到的是「只注册未建任务」用户的 token	total=0 空表（判据 total>0 方向对，但用错身份）	用例**自给自足**（自己上传数据集 + 建 h=7 任务），并拆成「≤1 s」与「非空表」两条断言		
T-10	verify_ep9_perf	用例间隐式依赖 case_main_path 内部创建的用户	失败归因困难	用例独立；并加 --only= 开关避免「改一条断言就整体重跑 5.7 分钟」		
T-11	verify_ep9_contract	未剥 client.ts 注释，注释里的反例 `http.get('/health')` 被当成真实调用点	凭空多出 GET /api/v1/health → 断言被自己的注释打败	解析前先剥注释；// 用 (?<![:\w]) 保护 https://		
T	verify_ep9_contract	parse_ts_union 只抓	`export type Horizon = 7 |	14 |	3	兼

#	脚本	缺陷	形态	处置		
-12		字符串字面量			0 解析成 []`	容数字字面量
T-13	verify_ep9_contract	断言「后端路由数 == 31」	把「API-01~API-30 编号数」跨口径当成「路由条目数」硬比（一个编号可落成多条路由）	改为**显式登记路由集合**比对（32 条），比数数更有证明力		
T-14	verify_ep9_contract	前端 EXPORT_COLUMNS 映射到后端**库列序**常量	同名不同义导致假失败	映射到 result_service.EXPORT_HEADER，并加「两者必须不同」的固化断言		
T-15	verify_ep9_browser	未设 VITE_API_TARGET，dev server 代理指向默认 8000 而后端在 8021	**整轮走查结论无效**（页面渲染空态，结果页文本长度仅 318）	显式设代理目标；并在断言中体现「结果页文本长度 4,005」这类**内容量**特征		
T-16	verify_ep9_browser	vite 只监听 IPv6（[::1]），127.0.0.1 不可达；且系统 HTTP_PROXY 会代理本地地址	页面全白、连接被代理截断	服务绑 127.0.0.1；浏览器加 --no-proxy-server		
T-17	verify_ep9_browser	首屏用 loadEventEnd 且后两次命中缓存，测出 41~48 ms	口径不当（load 事件早于 SPA 首屏绘制）	改用 **FCP**；冷加载与含缓存两次口径都记录		
T-18	verify_ep9_browser	没有「是否真的停在目标页」的断言	凭据失效时会静默跳 /login 而三条断言仍全绿	新增 18 条路径断言		
T-19	verify_ep9	一个集合同时服务「反向登记」与「幽灵条目」两种口径（按 verify_ 前缀过滤）	eval_report.py/self_check.py/a11y_audit.py 被误判为「清单有但磁盘没有」，一次 3 条假失败	拆成 audited（反向登记）与 all_names（幽灵条目）两个集合		

#	脚本	缺陷	形态	处置		
T-20	verify_ep9	§9.7 表格行数断言用「行首 + 词 + 空格」，实际匹配 0 行	断言失败却不给实际值	改「整格匹配」`\	\s*词\s*\	`，并把**实际匹配结果放进 detail**
T-21	verify_ep3_browser	dashboard 判据写死 header === 1	EP-9 全量回归实测为 **2** —— DashboardPage 自 EP-4 起用语义化 <header> 承载标题区，与 AppShell 顶栏合计 2 个，属**合理结构**（已用截图复核骨架完好）。原判据在 EP-3 后**未被任何走查覆盖过**，所以一直没暴露	判据改为「侧栏恰好 1 个 + 存在顶栏」，并保留上界 4 以防止「整页被塞进一个 <header>」这类真正的结构破坏；detail 输出全部计数便于人工复核		
T-22	verify_ep5_browser	结果页两条判据仍是 EP-5 时期的**临时占位断言**（要求 data-phase=succeeded 且页面显式标注「EP-6 补全」）	EP-6 落地后占位文案被移除、data-phase 语义由「任务终态字符串」改为页面就绪态 ready，原判据**必然失败** —— **断言的生命周期到了**，不是产品缺陷	升级为「EP-6 成果回归」：确认就绪态 + 确认真实指标（加权 MAPE）已渲染 + **反向**确认占位文案已清除（防止有人把占位加回来）		
T-23	verify_ep6_browser	① 硬编码依赖 TASK_ID（默认 19）指向一个**已存在的游客成功任务**；② 其夹具**不含 L2 序列**	① EP-9 跑在独立空库上 → 指标卡 0 张 + [FATAL] weighted_mape；② 补出任务后，因夹具 20 条全是 L1，「按 L2 过滤只返回 L2」用例拿到**空集**、断言恒假	新增 scripts/_ep9_make_fixture.py：造**游客**成功任务（必须 user_id=0，因走查全程**无 token**）+ 夹具含 **12 条 L2 序列**（本仓 50 天 < 90，跨仓聚合 150 ≥ 90）；用 TASK_ID=<n> 显式传入		

#	脚本	缺陷	形态	处置		
T-24	verify_ep3/4/5/6/7/8_browser	浏览器启动参数**缺 --no-proxy-server**	本机 HTTP_PROXY/HTTPS_PROXY 指向 127.0.0.1:51141，Edge 无头继承后**连本地地址也走代理** → 502 / 页面全白，极易误判成「前端崩了」（EP-9 踩坑，见 §1.2）	6 个脚本统一补 --no-proxy-server + --proxy-bypass-list=<-loopback>（与 verify_ep9_browser.mjs 对齐）		
T-25	verify_ep9_browser	首屏采样在 goto 后**只等 500 ms** 就读 paint timing	Chrome 实测 3 次只拿到 **2 个样本**（n=2）—— 冷加载时 React 还没把首屏画出来，getEntriesByType('paint') 是**空数组**。Edge 上侥幸 3/3 是它启动更快，属运气而非设计	给读取**加重试**（最多 6 × 400 ms，直到有值）。⚠ 刻意**没有**把断言放宽成 n≥2 —— 那等于自己削弱证据强度，比不修更糟		
T-26	verify_ep9_browser / _ep9_bidi_probe	finally 里 taskkill 之后**立刻** rmSync profile，异常被 catch 静默吞掉	两层原因：① spawn('taskkill') 是**异步**的，刚发出时浏览器子进程仍持有 profile 内的文件锁 → 删除必失败 → **永久残留**（这正是 P3-03 那 1.65 GB 的成因）；② 更隐蔽的一层 —— 平台的安全删除 hook 会把删除变成**异步的"移入回收站"**，于是即使删除**成功**，脚本刚退出时目录**仍在**，"删完立刻检查"会**误判成清理失败**（实测：走查结束即刻查到 1 个 79 MB，1 分钟后再查为 0）	改为「先等 1.5 s，再最多 6 次重试（每次间隔 1 s，以 existsSync 为准）」，并对仍失败者打印**显式告警**而非静默吞掉。验证方式：专门跑一轮走查到临时 OUT_DIR，**隔 1 分钟**再统计 —— 残留 0		

3.3 文档勘误类（3 项）

#	文档	原文	修正
D-01	技术文档 §9.4 B-03	举例值「2015/01/01、20150101」	→ 2015/13/45、20150103T99、未知、空串；并加 📌 勘误说明（PRD §3.2 字段表写的是「**推荐** YYYY-MM-DD」——推荐 ≠ 强制）

#	文档	原文	修正
D-02	技术文档 §9.4 `B-11` + `API-15-1` + §6.4	「L3 该序列**不写入** `predictions`」	→ 「只落 1 行占位（值=历史均值），**行数 ≠ horizon**；主指标不计入 L3」，并附冲突对照表与裁决依据
D-03	`models/prediction.py`	「导出列序由 `EXPORT_COLUMNS` 固定」	→ 明确其为**库列序**，导出表头在 `result_service.EXPORT_HEADER`（见 `F-04`）

---

4. 修复记录汇总

类别	数量	是否影响交付	处置状态
产品缺陷	5	`F-01` / `F-02` / `F-05` 影响功能正确性与安全口径；`F-03` 影响文档一致性；`F-04` 为误导性注释	☑ 全部修复并复验
校验脚本缺陷	25	不影响产品，但会掩盖真实问题（漏跑一次就少一次真缺陷被发现的概率）	☑ 全部修复并复验
文档勘误	3	影响评审依据	☑ 全部修正

复验方式：产品缺陷修复后，相关用例**重跑并单独确认**：

缺陷	复验用例	复验结果
F-01	`B-17`（7 条断言）	☑ 库中 `device='cpu'` 且 `device_fallback=1`
F-02	`B-20` / `B-21`（7 条断言）+ 上游 `verify_ep5_api` / `verify_ep6_api` / `verify_ep7_api` 的越权断言	☑ 三套脚本的越权断言口径已同步为 `40301`，全部通过

缺陷	复验用例	复验结果
F-03	`B-11`（14 条断言）	✅ 25/25 条 B 用例全通过
F-04	契约 D 组 3 条断言	✅ 92/92 通过
F-05	`verify_ep5_api` ⑨（4 条断言）+ `verify_ep9_api` 全量复跑	✅ ⑨ 转绿（`40301`）；`verify_ep9_api` 复跑 **125 / 0** 无回归

---

5. 遗留问题清单

分级：`P0` 阻塞交付 / `P1` 重要 / `P2` 二期 / `P3` 不排期。

本阶段无 `P0`。

P1（重要，需在 EP-10 说明或补救）

📌 `P1-01` 已在收尾补测中从「**未达成**」收敛为「**3 / 4 实测通过，余 1 属平台限制**」，

保留编号仅为与 `EP-7`/`EP-8` 时期的引用保持一致。

#	问题	现状	建议		
`P1-01`	`DOD-EP9` **第 4 条「四浏览器」：3/4 已实测通过，Safari 属平台限制**	**Edge 109/109、Chrome 109/109、Firefox 109/109** —— 三款均在 1366×768 与 1600×900 下无溢出/无空白	① 答辩时对 Safari 明确标注「平台限制，非项目缺陷」；如需覆盖，在 macOS 上跑同一脚本即可（用例已与内核解耦，Firefox 通道就是证明）；② 复跑命令 `BROWSER=<edge\	chrome\	firefox> node scripts/verify_ep9_browser.mjs，便携二进制在 data/tmp/browsers/`（获取方式见 §1.4，**不写注册表**）

#	问题	现状	建议		
		/无异常态/控制台 0 错误，各自出的报告见 §2.7。**Safari 无法实测**：Windows 平台没有 Safari 发行版 —— 这不是「没测」，是「测不了」			
P1-02	**极限规模（1,000 序列 × horizon=30）走软失败路径**	EP-9 实测 250 s → E_TIMEOUT。DOD-EP9 允许「通过或正确软失败」，五项判据均满足，故本阶段判定通过	若后续要求该规模也能在 60 s 内完成，需引入「按序列分批 + 单遍推理」或降低默认 horizon；当前设计选择「可预测地软失败」而非「不可预测地超时」，属有意取舍		

P2（二期）

#	问题	现状	建议
P2-01	B-10 的错误文案对二进制内容不够贴切	真 xlsx 改名 .csv 会在「大小/行数」阶段被拒，返回 40010（空文件），文案与「这其实是个 zip」的事实不符	第 2 步失败时探测文件魔数（PK\x03\x04），命中则提示「文件扩展名与内容不符，请勿仅改后缀」
P2-02	对比页 EP-9 只测了**空态**布局	对比页的选中任务存在前端 store（不落 URL），跨页刷新回到空态；选中态由 EP-7 的浏览器走查覆盖	若后续把选中任务写入 URL（?tasks=1,2），EP-9 的走查可一并覆盖选中态

P3（不排期）

#	问题	说明	
P3-01	前端构建产物存在 chunk 体积警告	EChart-CqH4xqeL.js 1,035 kB（gzip 344 kB）超过 500 kB 默认阈值。不影响功能（EP-8 起 ECharts 已懒加载），但建议后续用 manualChunks 拆分	
P3-02	verify_ep9_perf.py --only= 开关可产生「部分报告」	正式验收必须不带参数全跑（本轮已人工确认）；后续可在报告 JSON 里加 full_run: true/false 标记，让「部分运行」无法冒充全量	
~~P3-03~~	~~走查临时 profile 目录残留 1.65 GB~~ → ✅ **已解决（收尾补测时清零）**	outputs/ 下 .edge-profile* 目录 **32 → 0 个**；outputs/ep{3..9}-shots 合计 **≈1.94 GB → ≈35 MB**（28 张截图完好无损）。⚠ 统计时的坑：Chromium 给这些目录设了**隐藏属性**，Get-ChildItem 必须加 -Force，否则计数偏小、会误判「已删干净」	已为 verify_ep9_browser.mjs 补上模块作用域 profileDir + finally 清理。EP-9 之后跑的三款浏览器走查实测：每轮结束后 find 计数均为 **0**，确认**不再新增残留**

---

6. 证据文件清单

证据	路径
边界与异常（B-01~B-25）运行日志	控制台输出 125 项（临时库 data/tmp/verify_ep9_api/）
性能与集成实测数据	outputs/ep9_perf_report.json
契约一致性报告	outputs/ep9_contract_report.json
兼容性走查报告 + 截图（**3 款浏览器各一份**）	outputs/ep9-shots/ep9_browser_report{,_chrome,_firefox}.json、outputs/ep9-shots/{narrow,wide,chrome,firefox}/
Firefox BiDi 协议探针日志（协议差异的实测证据）	outputs/_ep9_bidi_probe.log（脚本 scripts/_ep9_bidi_probe.mjs；握手诊断 scripts/_ep9_bidi_diag.mjs）
浏览器便携二进制（**不写注册表、零系统污染**）	data/tmp/browsers/{chrome-win64,firefox}/（获取方式见 §1.4）
无障碍审计日志（退出码 0）	outputs/ep9_a11y_log.txt
校验链清单报告	outputs/ep9_chain_report.json
算法层自检日志	outputs/ep9_self_check.txt
端到端评估日志	outputs/ep9_eval_log.txt、outputs/eval_report.json
算法性能基线	outputs/eval_timing_last.json
上游阶段走查报告 + 截图（§2.11 全量实跑）	outputs/ep3-shots/browser_probe_summary.json、ep4-shots/ ~ ep8-shots/ 各自的报告 JSON + 截图
走查夹具生成脚本（§2.11 前置）	scripts/_ep9_make_fixture.py（游客成功任务：20 条 L1 + 12 条 L2 序列）

---

7. DOD-EP9 逐条对照

#	条目	判定	依据
1	🔒 端到端 ≤ 60 s（示例数据）；极限 1000 序列 × horizon=30 通过或正确软失败	☑	§2.6 第 1 项 11,855 ms；第 2 项软失败五项判据全满足
2	🔒 并发 3 个任务：全部成	☑	§2.6 第 3 项（32,207 /

#	条目	判定	依据
	功、无锁冲突、各自 ≤ 60 s		37,610 / 37,513 ms，锁冲突 0 次）
3	🔒 `a11y_audit.py` 退出码 0	☑	§2.8（53 项，FAIL 0）
4	四个浏览器 + 1366×768 无布局错乱	☑ **3 / 4 实测通过**	§2.7：**Edge / Chrome / Firefox 各 109/109**（1366×768 + 1600×900，控制台均 0 错误）；Safari 属 Windows 平台限制（`P1-01`）
5	浏览器控制台**无错误**（警告可有，错误不可有）	☑	§2.7（0 条，含未捕获异常）
6	§9.1 的 4 套校验脚本全部通过	☑	§2.1~2.4（`EP-9` 四套：边界 / 性能 / 兼容 / 无障碍）+ **§2.11**（上游 29 套全部**实跑**复绿），合计 35 个脚本（**该计数为 `EP-9` 收口时点**；`EP-10` 新增 `verify_docs_browser.mjs` / `verify_demo_data.py` / `verify_doc_exports.py` 后口径为 **38 套**，见实施方案 §12）
7	测试报告含：通过用例 / 失败用例 / 修复记录 / **遗留问题清单**	☑	本报告 §2 / §3 / §4 / §5

测试结论：`EP-9` 通过。 四项 🔒 硬约束全部达成；`DOD-EP9` 七条中六条**完全达成**，

第 4 条（四浏览器）为 **3 / 4 实测通过 + 1 项平台限制**（Safari 在 Windows 无发行版），

已如实登记为 `P1-01` 而非记作通过。项目具备进入 `EP-10`（文档与答辩交付）的条件。